

Here is a summary of the main points and takeaways from [lab1_text_data.ipynb](#):

1. The Limitation of "Bag-of-Words" Counting

The first half of the lab demonstrated that simply counting word frequencies is often insufficient for sentiment analysis.

- **The Baseline Problem:** You observed that a "majority class" baseline (predicting everything as negative) yielded ~66% accuracy. Your manual counting model struggled to beat this because common words overlap significantly across all sentiments.
- **Stopword Impact:** Removing the most frequent words (like "the" or "to") helps reduce noise, but if you remove too many or too few, the model loses its ability to distinguish between classes.

2. The Power of Text Preprocessing

You explored how cleaning data can change the "features" available to a model:

- **Tokenization:** Using specialized tools like TweetTokenizer is better than simple string splitting because it handles social media nuances like handles and punctuation.
- **Stemming vs. Lemmatization:** You saw that while Stemming (chopping off word ends) is fast, Lemmatization (finding the root dictionary word) is often more accurate, though it still misclassified several instances in this specific simple counting model.

3. Transitioning to Machine Learning

The final section introduced "Vectorization," which is the critical bridge between text and math.

- **TF-IDF (Term Frequency-Inverse Document Frequency):** You learned that this method weights words based on how unique they are to a specific tweet compared to the whole dataset.
- **Numerical Input:** You discovered that machine learning models like **Random Forest** cannot "read" text; they require these numerical vectors as input.
- **Performance Jump:** By using a Random Forest with vectorized text, your accuracy jumped from ~18% (with the counting model) to over **76%** on unseen test data.

4. Critical Concepts: Data Leakage & Overfitting

A major takeaway was learning how to properly evaluate a model:

- **Avoid Data Leakage:** You identified that splitting your data *after* building a vocabulary from the whole set is a mistake because it "leaks" information to the model.
 - **Train-Test Splits:** You saw that a model might achieve 99% accuracy on data it has already seen (training data) but perform significantly lower on data it hasn't seen (test data). This gap represents **overfitting**.
-